МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудов ого Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Программная инженерия

ОТЧЕТ

по лабораторной работе №8

по дисциплине «Введение в информационные технологии»

Тема: «Анализ и визуализация данных

использованием Pandas и Matplotlib»

Выполнил: студент группы БВТ2501

Лебедев Даниил Владимирович

Москва, 2026

Цель работы

Освоить основные приёмы анализа табличных данных с использованием библиотеки Pandas, научиться применять функции библиотеки NumPy при обработке данных, а также визуализировать результаты анализа с помощью библиотеки Matplotlib. Получить практические навыки работы с реальным набором данных и интерпретации результатов анализа.

Ход работы

0. Приложение

Для выполнения лабораторной работы используется стандартный набор данных Titanic (файл titanic.csv), содержащий информацию о пассажирах лайнера «Титаник».

```
// Зависимости
import pandas as pd
import numpy as np

// Считаем таблицу

df = pd.read_csv("Titanic-Dataset.csv")
```

```
print(df.head())
print(df.info())
print(df.describe())
```

```
                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
<class 'pandas.DataFrame'>
```

```
✓ [3] 261ms
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
```

```
None
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
```

Выведем отдельные столбцы и информацию об одной строке с помощью loc

```
print(df["Age"])
print(df[["Age", "Fare"]])
print(df.loc[0])
```

мы можем спокойно применять агрегатные функции из numpy к pandas

```
mean_age = np.mean(df["Age"].dropna())
max_fare = np.max(df["Fare"])
```

Также мы можем применять масочки как в numpy

```
adults = df[df["Age"] >= 18]
women = df[df["Sex"] == "female"]
```

Мы можем использовать group by как в sql, чтобы сгуппировать объекты по одному признаку

```
survival_by_sex = df.groupby("Sex")["Survived"].mean()
print(survival_by_sex)
```

Также графики

1) столбчатый

```
import matplotlib.pyplot as plt

survival_by_sex.plot(kind="bar")
plt.title("Доля выживших по полу")
plt.xlabel("Пол")
plt.ylabel("Доля выживших")
plt.show()
```

2) гистограмма

```
df["Age"].dropna().plot(kind="hist", bins=20)
plt.title("Распределение возраста пассажиров")
plt.xlabel("Возраст")
plt.ylabel("Количество пассажиров")
plt.show()
```

3) линейный график

```
fare_sorted = df.sort_values("Fare")
plt.plot(fare_sorted["Fare"].values)
plt.title("Изменение стоимости билетов")
plt.xlabel("Номер наблюдения")
plt.ylabel("Стоимость билета")
plt.show()
```

4) диаграмма рассеивания

```
plt.scatter(df["Age"], df["Fare"], alpha=0.5)
plt.title("Зависимость стоимости билета от возраста")
plt.xlabel("Возраст")
plt.ylabel("Стоимость билета")
plt.show()
```

5) две совмещёные гистограммы

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
df["Age"].dropna().plot(kind="hist", bins=20, ax=ax[0])
ax[0].set_title("Возраст")
df["Fare"].plot(kind="hist", bins=20, ax=ax[1])
ax[1].set_title("Стоимость билета")
plt.show()
```

Для удобства выполнял работу в jupiter notebook

1. Категориальные – числовые признаки

```
df.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Embarked'],
      dtype='str')
```

так как датасет мы загрузили выше воспользуемся маской для поиска

категориальных признаков

```
cat_features_mask = (df.dtypes == "str").values
cat_columns = df.columns[cat_features_mask]
cat_columns
```

```
Index(['Name', 'Sex', 'Ticket', 'Cabin', 'Embarked'], dtype='str')
```

но на самом деле это немного странный датасет и тут некоторые признаки с числовым значением мы можем назвать категориальными и наоборот поэтому набор категориальных признаков будет вот такой:

```
cat_columns = pd.Index(["Name", "Sex", "Pclass", "Embarked", "Survived"])
cat_features_mask = df.columns.isin(cat_columns)
cat_columns
```

```
Index(['Name', 'Sex', 'Pclass', 'Embarked', 'Survived'], dtype='str')
```

```
num_columns = df.columns[~cat_features_mask]
num_columns
```

числовые признаки

```
Index(['PassengerId', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare'], dtype='str')
```

2. масочки

масочка по полу

```
df_male = df[df["Sex"] == "male"]
df_female = df[df["Sex"] == "female"]
print(len(df_male), len(df_female))
```

вывод: 577; 314

масочка по возрасту

```
above_18 = df[df["Age"] >= 18]
under_18 = df[df["Age"] < 18]
print(len(above_18), len(under_18))
```

вывод: 601; 113

масочка по классу пассажира

```
print(len(above_18), len(under_18))
#%%
first_class = df[df["Pclass"] == 1]
second_class = df[df["Pclass"] == 2]
third_class = df[df["Pclass"] == 3]
print(len(first_class), len(second_class), len(third_class))
```

вывод: 216; 184; 491

То есть они практически ничем не отличаются от numpy

3. Средний возраст, пассажиров, максимальную и минимальную стоимость билета

```
avg_age_pd = df["Age"].mean()
avg_age_np = np.mean(df["Age"])
print(df["Age"].isna().sum())
print(avg_age_pd, avg_age_np)

max_ticket_price = df["Fare"].max()
min_ticket_price = df["Fare"].dropna().min()
print(max_ticket_price, min_ticket_price)
np_max_ticket_price = np.max(df["Fare"])
np_min_ticket_price = np.min(df["Fare"].dropna())
print(np_max_ticket_price, np_min_ticket_price)
```

```
177
29.69911764705882 29.69911764705882
512.3292 0.0
512.3292 0.0
```

функции работают абсолютно также как в numpy так как pandas основывается на numpy

4. Группировка

```
sex_class = df.groupby(['Sex', 'Pclass'])
survived_rate = sex_class["Survived"].mean()
avg_fare = sex_class["Fare"].mean()
avg_age = sex_class["Age"].mean()
print(survived_rate, avg_fare, avg_age)
```

```
Sex     Pclass
female  1         0.968085
        2         0.921053
        3         0.500000
male    1         0.368852
        2         0.157407
        3         0.135447
Name: Survived, dtype: float64 Sex     Pclass
female  1         106.125798
        2          21.970121
        3          16.118810
male    1          67.226127
        2          19.741782
        3          12.661633
```

```
Name: Fare, dtype: float64 Sex     Pclass
female  1         34.611765
        2         28.722973
        3         21.750000
male    1         41.281386
        2         30.740707
        3         26.507589
Name: Age, dtype: float64
```

так мы посчитали среднюю стоимость по сгруппированным значениям, чем-то они похожи на sql

5. Анализ null

для этого выведем все нулевые значения в строках

```
print(df.isnull().sum())
print("Всего пассажиров:", len(df))
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
Всего пассажиров: 891
```

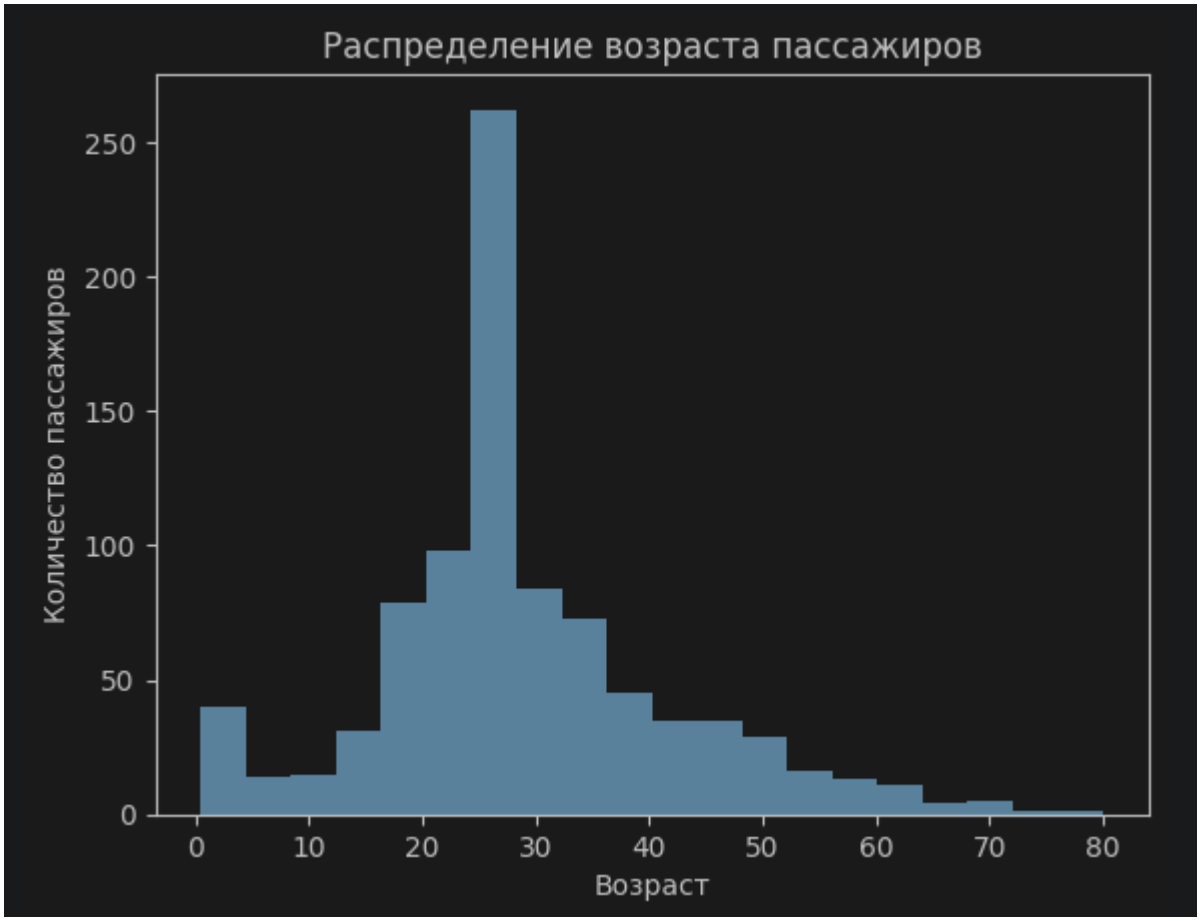
Значит Cabin выкидываем, а в Age на медиану, Embarked заменим на больший класс (моду)

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df = df.drop(['Cabin'], axis=1)
```

6. Графики

6.1

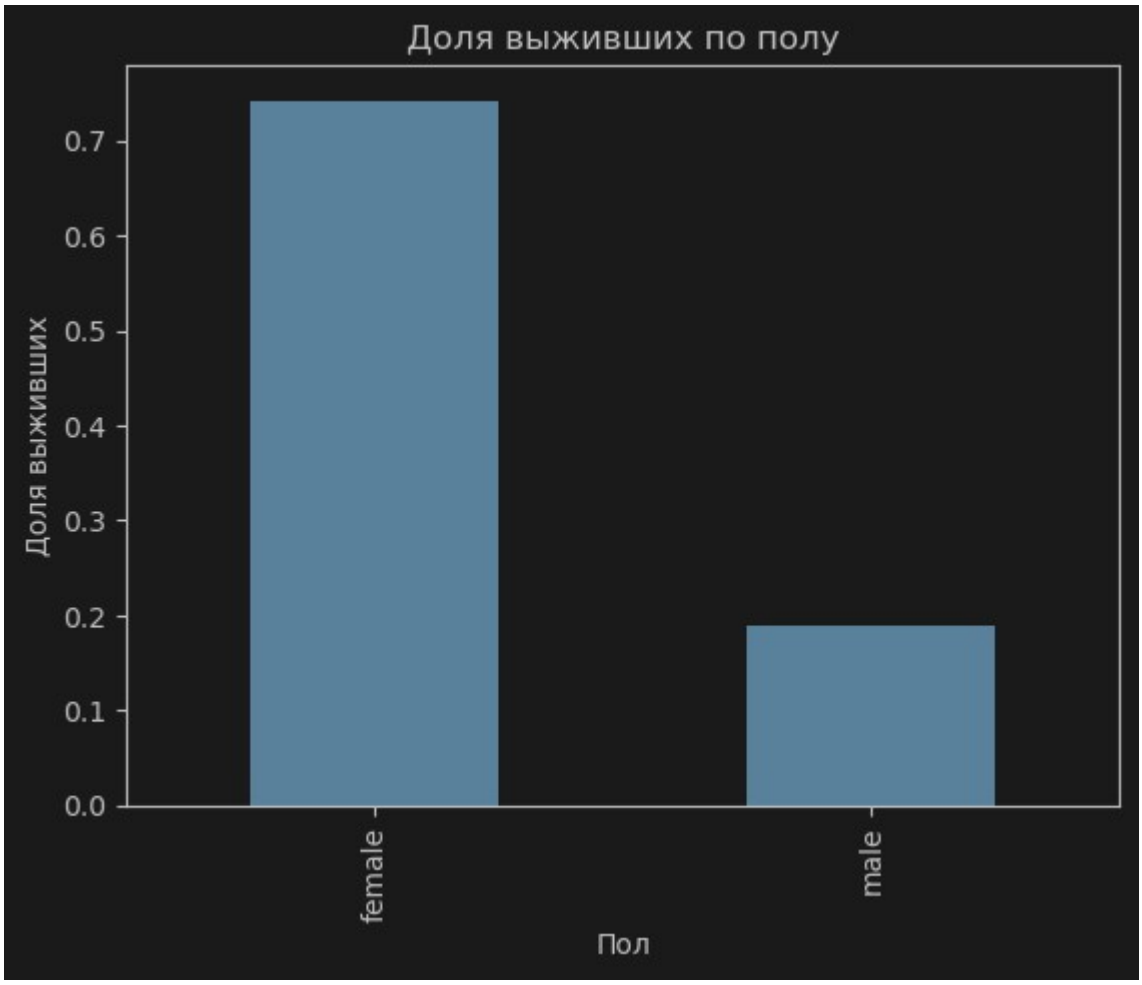
```
df["Age"].dropna().plot(kind="hist", bins=20)
plt.title("Распределение возраста пассажиров")
plt.xlabel("Возраст")
plt.ylabel("Количество пассажиров")
plt.show()
```



Выбросы: пассажиры 0 лет (их неоправданно много), основной диапазон значений 25-26 лет, так как туда попала медиана, которые мы назначили сторкам nan в графе возраст

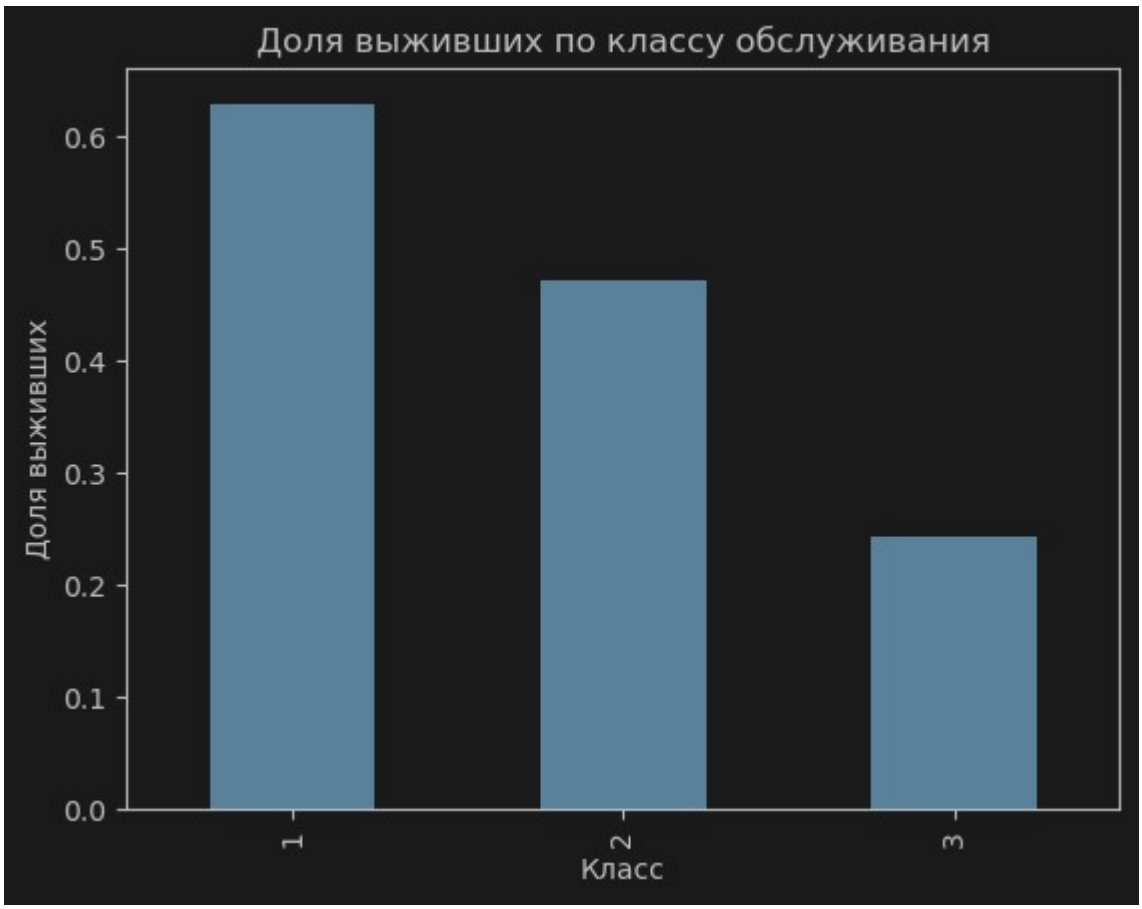
6.2

```
survival_by_sex.plot(kind="bar")
plt.title("Доля выживших по полу")
plt.xlabel("Пол")
plt.ylabel("Доля выживших")
plt.show()
```



Наблюдается резкая разница в выживаемости между женщинами и мужчинами. Доля выживших среди женщин составляет около **75%**, в то время как среди мужчин — менее **20%**. Данная визуализация подтверждает исторический факт соблюдения правила «сначала женщины и дети». У женщин шансы на спасение были почти в **4 раза выше**, чем у мужчин.

```
survival_by_class = df.groupby("Pclass")["Survived"].mean()
survival_by_class.plot(kind="bar")
plt.title("Доля выживших по классу обслуживания")
plt.xlabel("Класс")
plt.ylabel("Доля выживших")
plt.show()
```



Наблюдается четкая зависимость между классом обслуживания и шансом выжить. Чем выше класс (меньше число), тем выше вероятность спасения.

Заключение:

мы научились применять функции библиотеки NumPy при обработке данных, а также визуализировать результаты анализа с помощью библиотеки Matplotlib. Получили практические навыки работы с реальным набором данных и интерпретации результатов анализа.

Гитхаб
https://github.com/Bulbanator123/labsmtucivvit